

Hierarchical design with VHDL

Lecture 2

10100101010011110100001001011101001011010101011101000041000101010100
0041000010100101001001010000101101001010140000111101001010100111101000010010111010010
110101010101110100004100001010010100101000010110100101014000011110100101

Electronic and Communication Engineering
Department

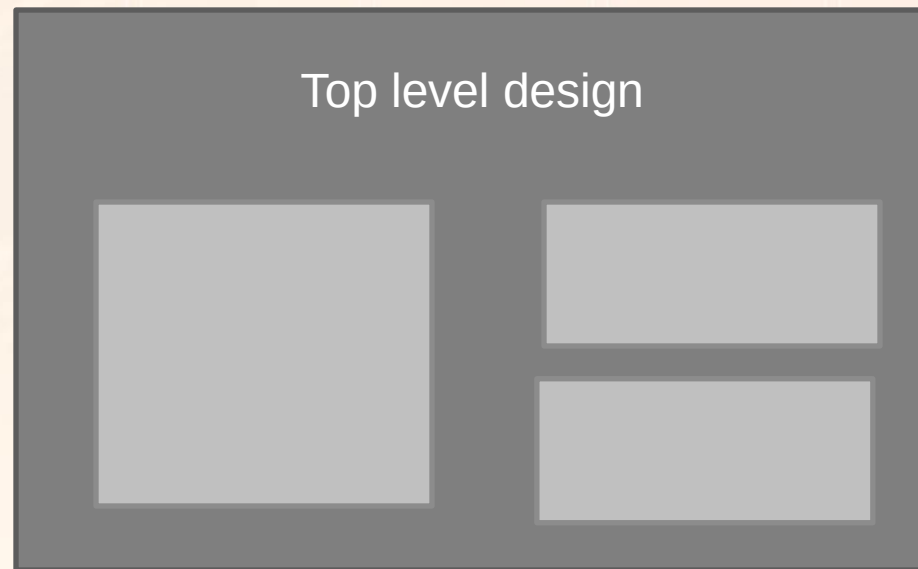
ECE 307 Introductory to Digital Systems Lab.

Dr. Ammar Jallawi Mahmood Karkar

Ammar.karkar@uokufa.edu.iq

Why hierarchical design

- Ability to reuse common elements
- Segment a complex design into smaller pieces
- More maintainable design



Complexity management

- *Focus on a manageable portion of the system, and analyze, design and verify each module in isolation.*
 - *Divided-and-conquer strategy*
- *Construct the system in stages by a designer or concurrently by a team of designers.*
- *Help synthesis process*

Design reuse

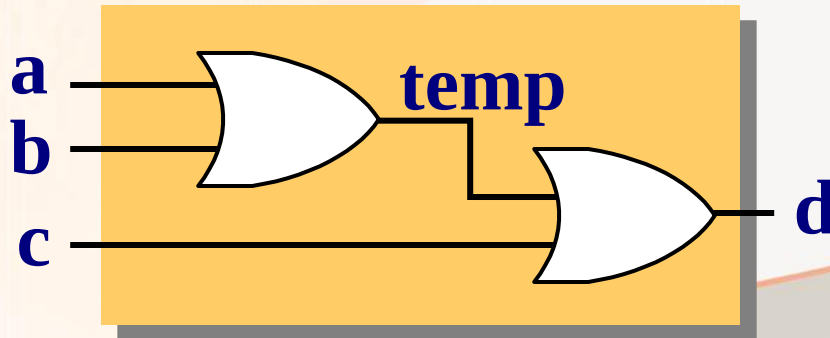
- *Use predesigned modules or third-party cores*
- *Use the same module in different design*
- *Isolate device-dependent components (e.g., SRAM)*

Components

- *Hierarchical design usually shown as a block diagram (structural description)*
- *VHDL component is the mechanism to describe structural description in text*
- *To use a component:*
 - *Component declaration (make known)*
 - *Component instantiation (create an instance)*

Hierarchy

- *Entities can be pieced together to form larger structures*
- *The interconnection of the entities and their architectures describe the **hierarchy** of a design*
- *A design entity **instantiates** lower level design entities, referred to as **components***
- *As an example, consider the construction of a 3-input OR gate from 2-input OR gates used as components*



The 2-input OR gate

```
entity or_gate is
    port(a, b: in std_logic;
          c: out std_logic);
end or_gate;

architecture structural of or_gate is
begin
    c <= a or b;
end structural;
```

- *To use an entity (e.g, the OR gate above) as a component within another entity, two steps are required:*
 - **Component declaration** – the components that will be used within the entity must be declared, in the same way and in the same place as internal signals. Each component is declared once.
 - **Component instantiation** – the component must be placed in the architecture and connected to the design. The same component can be instantiated multiple times.

The 3-input OR gate

```
library ieee;
use ieee.std_logic_1164.all;
-----
entity top is
    port(a: in std_logic;
          b: in std_logic;
          c: in std_logic;
          d: out std_logic);
end top;
-----
architecture structural of top is
    component or_gate is
        port(a, b: in std_logic;
              c: out std_logic);
    end component;
    signal temp: std_logic;
begin
    or1: or_gate
        port map (a => a, b => b, c => temp);
    or2: or_gate
        port map (a => temp, b => c, c => d);
end structural;
```

Component declaration:
very similar to entity
declaration – cut & paste

Component instantiation:
creates an **instance** of the
component, identified by a
unique label. The I/O **ports** of
the component must be
mapped to internal or I/O
signals in the architecture.

The 3-input OR gate

```
library ieee;
use ieee.std_logic_1164.all;
-----
entity top is
    port(a: in std_logic;
         b: in std_logic;
         c: in std_logic;
         d: out std_logic);
end top;
-----
architecture structural of top is
    component or_gate is
        port(a, b: in std_logic;
             c: out std_logic);
    end component;
    signal temp: std_logic;
begin
    or1: or_gate
        port map (a => a, b => b, c => temp);
    or2: or_gate
        port map (a => temp, b => c, c => d);
end structural;
```

Note the syntax to **map** a component's I/Os to internal signals: keyword **PORT MAP** followed by a list of mappings in the form

IO => internal_signal

Note particularly the use of “=>”, the opposite of an assignment

Component Instantiation

```
library ieee;
use ieee.std_logic_1164.all;
-----
entity top is
    port(a: in std_logic;
         b: in std_logic;
         c: in std_logic;
         d: out std_logic);
end top;
-----
architecture structural of top is
    component or_gate is
        port(a, b: in std_logic;
             c: out std_logic);
    end component;
    signal temp: std_logic;
begin
    or1: or_gate
        port map (a => a, b => b, c => temp);
    or2: or_gate
        port map (a => temp, b => c, c => d);
end structural;
```

Component declaration:
very similar to entity
declaration – cut & paste

Component instantiation:
creates an instance of the
component, identified by a
unique label.

Direct Instantiation

```
library ieee;
use ieee.std_logic_1164.all;
-----
entity top is
    port (a: in std_logic;
          b: in std_logic;
          c: in std_logic;
          d: out std_logic);
end top;
-----
architecture structural of top is
    signal temp: std_logic;
begin
    or1: entity work.or_gate
        port map (a => a, b => b, c => temp);
    or2: entity work.or_gate
        port map (a => temp, b => c, c => d);
end structural;
```

No component declaration:
relies on presence of **work** library

Component instantiation:
creates an **instance** of the component, identified by a **unique label**. Must specify that the component is to be found within the **work** library.

PREFERRED

A word of caution...

- *Some texts (and many websites) propose an alternate notation for port maps, which does not involve the explicit definition of I/Os*

```
architecture structural of top is
    signal temp: std_logic;
begin
    or1: entity work.or_gate
        port map (a, b, temp);
    or2: entity work.or_gate
        port map (temp, c, d);
end structural;
```

The order of the I/Os must be the same as in the component declaration

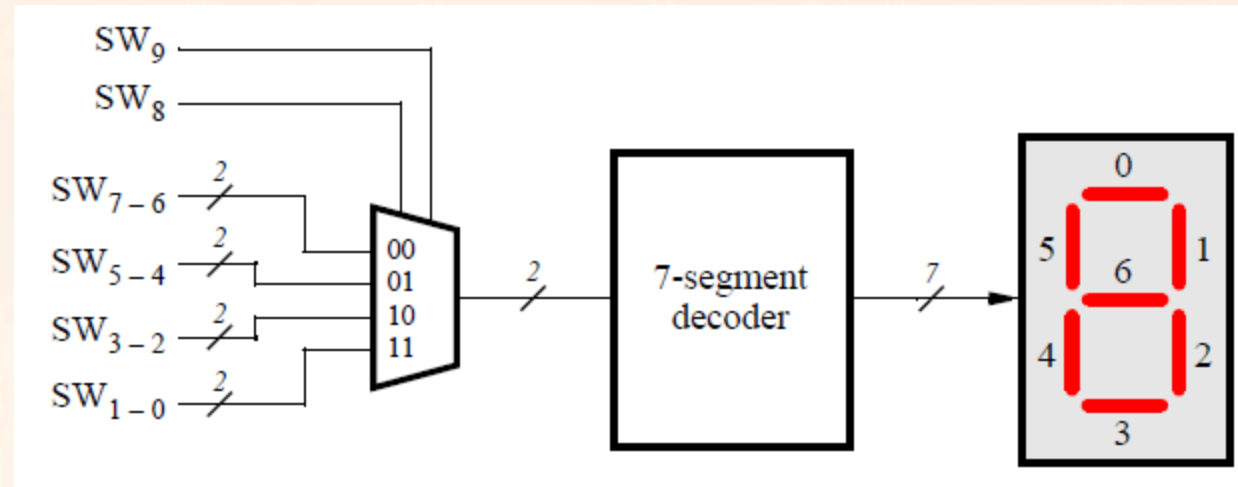
BAD

- *This is NOT a good idea (I/Os can be changed as the circuit is designed and tested) and should be avoided (particularly in direct instantiation)*

Example

- two-bit wide 4-to-1 multiplexer to enable the selection of four characters that are displayed on a 7-segment display

M1	M0	
0	0	d
0	1	E
1	0	1
1	1	



Component declaration

ARCHITECTURE Structure OF part5 IS

COMPONENT mux_2bit_4to1

PORT (S, U, V, W, X : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
M : OUT STD_LOGIC_VECTOR(1 DOWNTO 0));

END COMPONENT;

COMPONENT char_7seg

PORT (C : IN STD_LOGIC_VECTOR(1 DOWNTO 0);
Display : OUT STD_LOGIC_VECTOR(0 TO 6));

END COMPONENT;

SIGNAL Ch_Sel, Ch0, Ch1, Ch2, Ch3 :

STD_LOGIC_VECTOR(1 DOWNTO 0);

SIGNAL H3_Ch, H2_Ch, H1_Ch, H0_Ch :

STD_LOGIC_VECTOR(1 DOWNTO 0);

BEGIN

Component instantiation

- *Create an instance of a component*
- *Map formal signals to actual signals*

```
M3: mux_2bit_4to1 PORT MAP (Ch_Sel, Ch0, Ch1, Ch2, Ch3, H3_Ch);  
M2: mux_2bit_4to1 PORT MAP (Ch_Sel, Ch1, Ch2, Ch3, Ch0, H2_Ch);  
M1: mux_2bit_4to1 PORT MAP (Ch_Sel, Ch2, Ch3, Ch0, Ch1, H1_Ch);  
M0: mux_2bit_4to1 PORT MAP (Ch_Sel, Ch3, Ch0, Ch1, Ch2, H0_Ch);
```

```
H3: char_7seg PORT MAP (H3_Ch, HEX3);  
H2: char_7seg PORT MAP (H2_Ch, HEX2);  
H1: char_7seg PORT MAP (H1_Ch, HEX1);  
H0: char_7seg PORT MAP (H0_Ch, HEX0);
```

Order of signal should be the same as of the ports in declaration in of the entity

Conclusions

- *Hieratical design make designer job much easier.*
- *Now let us finish lab 1*

Thank you!

